

Veri Yapıları Lab 2

Pointer

Bir değişkenin doğrudan değerini değil, bellekte tutulduğu **adresini** saklayan özel değişkenlerdir.

Neden Kullanılır?

- Dinamik bellek yönetimi için
- Fonksiyonlara değer yerine referans (adres) aktararak orijinal veriyi değiştirebilmek için (pass by reference)
- Diziler ve stringler üzerinde daha verimli işlemler yapabilmek için
- **&** (Address-of): Değişkenin bellekteki **adresini** döndürür.
- ***** (Dereference): Pointer'ın gösterdiği adresteki **değere** erişmeyi sağlar.

```
int sayi = 10;  
int *ptr = &sayi; // sayi değişkeninin adresi ptr'ye atanır  
  
printf("%d", *ptr); // Ekran çıktısı: 10 (adresteki değere ulaşıldı)
```

Generic (Genel) Pointer (void *)

Herhangi bir veri türünün (`int`, `float`, `char` veya `struct`) adresini tutabilen, tür bağımsız işaretçilerdir.

Temel Özellikler:

- **Türsüzlük:** Doğrudan herhangi bir veri tipine atanabilir; ancak derleyici gösterilen verinin boyutunu bilmediği için **doğrudan dereference (*ptr) edilemez**.
- **Tip Dönüşümü (Casting):** Kullanılacağı zaman hedef veri türüne açıkça dönüştürülmesi gerekir.
- **Pointer Aritmetiği:** Standart C'de `void *` üzerinde aritmetik işlemler (`ptr++` vb.) yapılamaz.

Örnekler (Generic Pointer)

```
int sayi = 42;  
void *ptr = &sayi; // Herhangi bir türün adresi atanabilir  
  
// Kullanırken (dereference ederken) tür dönüşümü yapılmalıdır:  
printf("%d", *(int *)ptr);
```

```
int dizi[ ] = {10, 20, 30};  
void *ptr = dizi; // Dizinin başlangıç adresi  
  
// 1 int eleman ileri gitmek için (int 4 bayt ise 4 bayt ilerletilir):  
ptr = (char *)ptr + sizeof(int);  
  
// Veya sadece 1 bayt ileri gitmek için:  
ptr = (char *)ptr + 1;
```

Özellik	Normal Pointer (int* , char* vb.)	Generic Pointer (void*)
Tür Bağlantısı	Belirli bir veri tipine sıkı sıkıya bağlıdır.	Tip bağımsızdır, her türün adresini tutabilir.
Dereference (*)	Doğrudan yapılabilir (*ptr ile değere erişilir).	Boyut bilinmediği için doğrudan yapılamaz , casting şarttır (*(int*)ptr).
Pointer Aritmetiği	Güvenle yapılabilir (ptr++ bir sonraki veriye gider).	Standart C'de boyutu belirsiz olduğu için desteklenmez .
Kullanım Amacı	Tip güvenli (type-safe) değişken ve bellek yönetimi.	malloc() veya qsort() gibi çok biçimli (generic) fonksiyonlar.

-> (Ok) Operatörü

Struct (yapı) türündeki değişkenlere normalde nokta (.) operatörü ile erişilir (örn: `ogrenci.yas`).

Ancak elinizde struct'ın kendisi değil de **adresini tutan bir pointer** varsa, elemanlara erişmek için özel bir operatöre (->) ihtiyaç duyulur.

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main(void) {
    int* newData;
    int* nodeData;

    NODE* node;

    newData = (int*)malloc(sizeof(int));

    *newData = 7;

    node = createNode(newData);
    nodeData = (int*)node->dataPtr;
    printf ("Data from node: %d\n", *nodeData);

    return 0;
}
```

Sıra Sizde;

1. Soru

Kullanıcının yaşını (23 gibi bir tamsayı) bilgisayar belleğinde dinamik olarak yer açarak saklayan ve bunu bir düğüm yapısı üzerinden okuyup ekrana yazdıran bir program yazınız.

2. Soru

Bir ürünün fiyatını (49 . 99 gibi ondalıklı bir sayı) tür bağımsız bir yapı kullanarak bellekte tutan ve düğüm içerisindeki bu değeri ekranda gösteren bir C kodu yazınız.

3. Soru

Bir öğrencinin ders başarı harfini (' A ' gibi bir karakter) dinamik bellek yardımıyla oluşturup bir düğüme bağlayan, ardından bu değere ok operatörüyle erişerek ekrana yazdıran bir program yazınız.